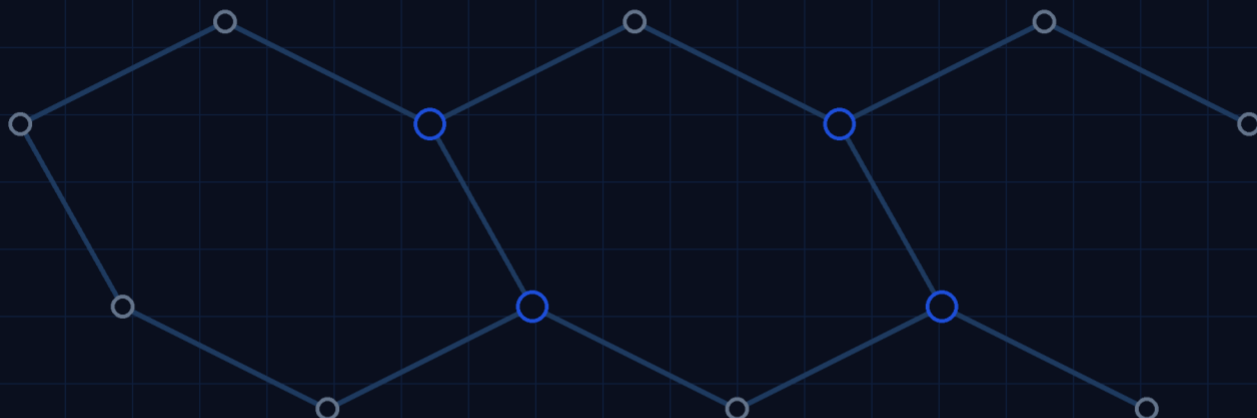


ARKHEION



ARKHEION

SOVEREIGN INFRASTRUCTURE

Kubernetes and Spring Boot for EVM

EVM smart contract cluster orchestration

EVM 智能合約集群編排基礎設施

TECHNICAL PRODUCT WHITEPAPER

技術產品白皮書

Version 1.0 / May 2026

Open-source CLI core / Enterprise infrastructure roadmap

Arkheion: Governed Infrastructure for EVM Contract Clusters

Arkheion : 面向 EVM 智能合約集群的治理基礎設施

Kubernetes and Spring Boot for EVM

Version 1.0 / May 2026 / Bilingual edition

Document purpose. This whitepaper introduces Arkheion as an open-source smart contract cluster orchestration framework and describes the enterprise infrastructure direction built around the same core. It is a technical product document, not an investment prospectus.

文件用途：本白皮書介紹 Arkheion 開源智能合約集群編排框架，以及圍繞同一內核構建的企業級基礎設施方向。本文是技術產品文件，不構成投資說明書。

Status and evidence boundary

狀態與證據邊界

The available implementation includes a working CLI, EVM contracts, lifecycle commands, checkpoints, audit-oriented logs, cluster locking, topology analysis, upgrade and rollback paths. A 20-module DeFi sample cluster has completed local-chain end-to-end validation. The Gas results in this paper come from the reproducible benchmark under gas-benchmark/.

當前實現包含可執行的 CLI、EVM 合約、生命週期命令、檢查點、面向審計的日誌、集群鎖、拓撲分析、升級與回滾路徑。20 模組 DeFi 樣板集群已經完成本地鏈端到端驗證。本文 Gas 資料來自 gas-benchmark/ 下可復現的 benchmark。

Public testnet hardening, dual-account multisignature acceptance testing, HSM or KMS integration, enterprise approval workflows, and a hosted control plane are roadmap items. They are not presented as delivered capabilities.

公共測試網強化、雙帳號多籤驗收、HSM 或 KMS 對接、企業審批流以及託管控制台屬於路線圖事項，本文不將其表述為已經交付的能力。

A separate enterprise codebase contains the Rust control-plane foundation: arkctl, an execution-plan DSL, preflight and ledger flow, task-runtime primitives, permission checks, Remote Signer interfaces, compiler foundations, and PostgreSQL migration drafts. Production handlers, reconciliation, Dashboard delivery and full integration remain in progress.

獨立的企業版程式庫已包含 Rust 控制面基礎：arkctl、執行計畫 DSL、preflight 與命令帳本流程、任務執行時原語、權限檢查、Remote Signer 介面、編譯器基礎與 PostgreSQL 遷移草案。生產處理器、對帳、Dashboard 交付與完整整合仍在推進中。

CONTENTS

目錄

Section	English	繁體中文
01	Executive summary	執行摘要
02	Why institutions need Arkheion	金融機構為何需要 Arkheion
03	Complete capability map	完整能力地圖
04	On-chain architecture	鏈上架構
05	Security model	安全模型
06	Declarative assembly and lifecycle	宣告式裝配與生命週期
07	Gas benchmark	Gas 基準測試
08	Enterprise control plane	企業控制面
09	Product editions and adoption	產品版本與採用路徑
10	Scope, limitations and roadmap	邊界、限制與路線圖
11	Getting started and references	快速開始與參考資料

01 Executive summary

執行摘要

Arkheion is a smart contract cluster orchestration framework for EVM systems. It treats an application as a governed set of independently deployed modules rather than a single contract or a shared-storage proxy. Each module retains its own storage boundary while participating in an inspectable runtime topology.

Arkheion 是面向 EVM 系統的智能合約集群編排框架。它將應用視為一組受治理、可獨立部署的模組，而不是單一合約或共享儲存代理。每個模組保留自身儲存邊界，同時參與可檢查的執行時拓撲。

The product idea can be summarized in one sentence: Kubernetes and Spring Boot for EVM. Kubernetes contributes the operational model of clusters, lifecycle management and controlled change. Spring Boot contributes a developer experience centered on declarative configuration and a repeatable assembly flow. Arkheion adapts these ideas to the constraints of deterministic on-chain execution.

產品理念可以用一句話概括：Kubernetes and Spring Boot for EVM。Kubernetes 提供集群、生命週期管理和受控變更的運維模型；Spring Boot 提供以宣告式配置和可重複裝配流程為中心的開發體驗。Arkheion 將這些思想適配到確定性的鏈上執行環境。

Arkheion manages a system of contracts, the process of change, and the evidence required to explain that change.

Arkheion 管理的是一組合約、一次變更的完整過程，以及解釋該變更所需的證據。

Two product layers

兩層產品形態

Layer	Available today	Direction
Open-source core 開源內核	CLI, EVM contracts, auto assembly, topology, upgrade, rollback, checkpoints, logs	Developer adoption and reproducible engineering workflows 開發者採用與可復現工程流程
Enterprise infrastructure 企業級基礎設施	Built on the same core architecture 基於同一核心架構	SaaS and private deployment, policy controls, approval workflow, custody integration, reporting SaaS 與私有化部署、策略、審批、託管集成和報告

02 Why institutions need Arkheion

金融機構為何需要 Arkheion

As on-chain applications grow, the main engineering challenge shifts from deploying one contract to governing a system over time. Financial applications amplify this problem because topology, permissions, upgrade history and failure recovery become part of the operational risk surface.

隨着鏈上應用變複雜，核心工程挑戰會從部署一個合約轉向長期治理一個系統。金融應用進一步放大了這一問題，因為拓撲、權限、升級歷史和故障恢復都會成為運營風險面的一部分。

Why contract clusters need an operating model

為什麼合約集群需要操作模型

- A multi-module protocol has dependencies that must remain explainable after deployment.
多模組協議在部署後仍需要保持依賴關係可解釋。
- A change to one module can affect callers, permissions, and recovery options.
單個模組變更可能影響呼叫方、權限以及恢復選項。
- Deployment scripts can send transactions, but they do not automatically create an auditable lifecycle.
部署腳本可以發送交易，但不會自動形成可審計的生命週期。
- Financial institutions require controlled change, evidence retention, and clear responsibility boundaries.
金融機構要求受控變更、證據留存以及清晰的責任邊界。

A transaction hash is not an operating model

交易雜湊不是營運模型

A transaction hash can prove that a state transition happened. It does not prove that the change was planned, reviewed, simulated, approved under separation-of-duties rules, reconciled with off-chain records, or recoverable after partial failure. For regulated digital-asset operations, those missing controls are the actual infrastructure gap.

交易雜湊可以證明狀態轉移已發生，但無法證明該變更經過規劃、覆核、模擬、職責分離下的審批、鏈下記錄對帳，或在部分失敗後仍可恢復。對受監管的數位資產營運而言，這些缺失的控制才是真正的基礎設施缺口。

- Change control: explain what will change before any privileged transaction is sent.
變更控制：在發送任何高權限交易前，說明將要發生的變更。
- Separation of duties: distinguish planning, approval, signing, execution and observation.
職責分離：區分規劃、審批、簽名、執行與觀察。
- Reconciliation: detect divergence between on-chain state and institutional records.
對帳：偵測鏈上狀態與機構記錄之間的差異。
- Recovery: return a cluster to a known safe state after interruption or unsuitable change.
恢復：在中斷或不適當變更後，將集群恢復至已知安全狀態。

Why compare with Diamond

為什麼與 Diamond 對比

Diamond (EIP-2535) is the relevant architectural comparison for this paper because it offers modular upgradeability through facets behind one proxy address. Arkheion chooses a different tradeoff: independently deployed modules, runtime dependency resolution, and explicit topology management. The benchmark section measures the consequences directly.

Diamond (EIP-2535) 是本文最相關的架構對比對象，因為它通過單一代理地址後的 Facet 提供模組化升級能力。Arkheion 選擇了不同的權衡：獨立部署模組、執行時依賴解析以及顯式拓撲管理。後文 benchmark 會直接量化這種差異。

03 Complete capability map

完整能力地圖

Arkheion covers the contract-cluster lifecycle from developer bootstrap to controlled recovery. The open-source CLI is intentionally usable on its own: institutions can inspect the operating model before evaluating enterprise integration.

Arkheion 覆蓋合約集群從開發者初始化到受控恢復的生命週期。開源 CLI 本身即可使用，機構可以先檢視其營運模型，再評估企業整合。

Domain	Open-source capabilities	Institutional value
Bootstrap 初始化	init, environment scaffolding, cluster skeleton	Repeatable project baseline 可重複的專案基線
Assembly 裝配	check, auto --dry-run, auto, deploy, mount, unmount, link, unlink	Reviewable topology change 可覆核的拓撲變更
Governance 治理	wallet owners, submit, confirm, execute, revoke, propose; normal rights	Explicit privileged paths 明確的高權限路徑
Versioning 版本治理	upgrade, rollback, generation, deploySeq, podSnapshot	Explainable recovery 可解釋的恢復
Reliability 可靠性	txExecutor, clusterLock, checkpoint resume or restart	Controlled interruption handling 受控的中斷處理
Evidence 證據	graph, history, project state and logs	Audit-trail foundation 審計留痕基礎
Compiler foundation 編譯器基礎	Standard JSON pipeline and artifact metadata direction	Deployability and evidence checks 可部署性與證據檢查

04 On-chain architecture

鏈上架構

Arkheion separates business logic from orchestration responsibilities. The CLI prepares and executes lifecycle operations. The control plane coordinates registration, topology and permissions. Business modules inherit a common template while preserving independent contract storage.

Arkheion 將業務邏輯與編排職責分離。CLI 負責準備並執行生命週期操作；控制面協調註冊、拓撲和權限；業務模組繼承統一模板，同時保留獨立的合約儲存。

Layer	Core elements	Responsibility
Developer interface 開發者接口	arkheion CLI	init, check, auto, deploy, graph, history, upgrade, rollback
Control plane 控制面	ClusterManager EvokerManager ProxyWallet MultiSigWallet	Registry, topology coordination, permission routing, governance
Business modules 業務模組	NormalTemplate Pods	Isolated storage, lifecycle hooks, authenticated cross-module calls
Evidence layer 證據層	project.json logs/ checkpoint files	Local state, audit trail, interruption recovery

Pod as the deployment unit

Pod 作為部署單元

A Pod is an independently deployed contract module. It is not a container and does not reproduce Kubernetes mechanically. The term captures the operating model: modules are registered, linked, mounted, inspected, upgraded and rolled back as governed units.

Pod 是獨立部署的合約模組。它不是容器，也不是對 Kubernetes 的機械複製。這個術語表達的是一種操作模型：模組以受治理單元的方式被註冊、連結、掛載、檢查、升級和回滾。

Control plane responsibilities

控制面職責

- ClusterManager maintains registry-level identity and orchestration entry points.
ClusterManager 維護註冊級身份和編排入口。
- EvokerManager coordinates the runtime dependency graph.
EvokerManager 協調執行時依賴圖。
- NormalTemplate gives modules lifecycle hooks and verification primitives.
NormalTemplate 為模組提供生命週期鉤子和驗證原語。
- Wallet components route privileged operations through explicit governance paths.
錢包組件將高權限操作路由到顯式治理路徑。

05 Security model

安全模型

Arkheion does not use Pod edges as a convenience-only service discovery layer. activePod and passivePod are primarily security primitives. They express which module expects to call another module and which module agrees to accept that relationship.

Arkheion 並不把 Pod 邊僅僅作為便捷的服務發現層。activePod 與 passivePod 首先是安全原語：它們表達了哪個模組預期呼叫另一個模組，以及哪個模組同意接受這種關係。

Bilateral dependency intent

雙向依賴意圖

Primitive	Meaning	Security role
activePod	Outgoing dependency intent 出向依賴意圖	The caller declares the module it expects to use 呼叫方聲明預期使用的模組
passivePod	Incoming dependency intent 入向依賴意圖	The callee records the module relationship it accepts 被呼叫方記錄接受的模組關係
podSnapshot	Topology snapshot by contract ID 按合約 ID 保存的拓撲快照	Supports consistent local state and address-independent recovery 支持一致的本地狀態與地址無關恢復

Storage isolation

儲存隔離

Each Arkheion module is an independent contract with its own storage. This costs more during deployment, rewiring and cross-Pod execution than shared-storage designs in some scenarios. The benefit is a clear storage boundary: an error in one module does not silently overwrite another module through a shared delegatecall storage namespace.

每個 Arkheion 模組都是擁有獨立儲存的合約。在部分場景中，這會比共享儲存設計產生更高的部署、重連和跨 Pod 執行成本。其收益是清晰的儲存邊界：單個模組的錯誤不會通過共享 delegatecall 儲存命名空間靜默覆蓋其他模組的資料。

Operational controls

操作控制

- Safety gates and confirmations precede intrusive commands.
侵入式命令執行前設置安全閘門和確認機制。
- clusterLock reduces conflicting local command execution.
clusterLock 降低本地命令併發衝突。
- txExecutor centralizes transaction execution behavior.
txExecutor 統一交易執行行為。
- Logs and project state are part of the evidence trail.
日誌與項目狀態共同構成證據鏈。

06 Declarative assembly and lifecycle

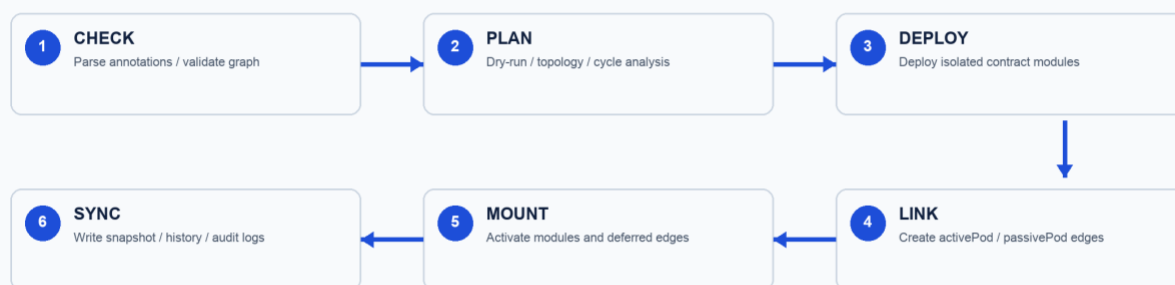
宣告式裝配與生命週期

Arkheion uses lightweight source annotations and a staged CLI pipeline to assemble a contract cluster. The goal is not to hide chain behavior. The goal is to make the planned behavior reviewable before transactions are sent and recoverable if execution is interrupted.

Arkheion 使用輕量級源碼註解和分階段 CLI 流水線裝配合約集群。目標不是隱藏鏈上行為，而是讓計劃在交易發出前可審閱，並讓執行中斷後可以恢復。

DECLARATIVE AUTO ASSEMBLY

宣告式自動裝配流程



Safety boundary: state changes are committed with checkpoint, podSnapshot consistency and logs.
安全邊界：鏈上變更與 checkpoint、podSnapshot 一致性及日誌留痕同步提交。

Figure 1. Arkheion declarative auto assembly flow / 圖 1 : Arkheion 宣告式自動裝配流程

Developer-facing commands

面向開發者的命令

```
arkheion cluster check
arkheion cluster auto --dry-run
arkheion cluster auto
arkheion cluster graph && arkheion cluster history
```

Dependency analysis

依賴分析

The analysis stage validates IDs and topology, identifies Pod-level cycles, detects convention-based function-call cycles, and separates safe deferred edges from edges that should be skipped for review. The function-level analysis is intentionally lightweight and is not a sound whole-program Solidity verifier.

分析階段驗證 ID 與拓撲，識別 Pod 級循環，基於約定檢測函數呼叫環，並將可安全延遲的邊與應跳過並交由人工審閱的邊區分開。函數級分析刻意保持輕量，不是具有完備性的 Solidity 全程序驗證器。

Lifecycle governance

生命週期治理

A production-oriented system must explain not only the current address of a module, but also how it reached that address and how it can recover from a failed or unsuitable change. Arkheion models these questions as lifecycle operations.

面向生產的系統不僅要解釋模組當前地址，還要解釋它如何到達該地址，以及如何從失敗或不合適的變更中恢復。Arkheion 將這些問題建模為生命週期操作。

Capability	Current open-source behavior	Institutional relevance
Dry-run 預演	Produces an execution plan without chain mutation 輸出計劃但不修改鏈上狀態	Pre-change review 變更前複覈
Checkpoint 檢查點	Supports resume or restart for staged operations 支持分階段操作 resume 或 restart	Controlled recovery from interruption 中斷後的受控恢復
History 歷史	Tracks generation and deployment sequence 記錄 generation 和 deploySeq	Explainable module evolution 可解釋的模組演進
podSnapshot 拓撲快照	Synchronizes ID-based topology after state changes 狀態變更後同步基於 ID 的拓撲	Rollback-oriented consistency 面向回滾的一致性
Logs 日誌	Writes command evidence when project state exists 存在項目狀態時寫入命令證據	Audit trail foundation 審計留痕基礎

Upgrade and rollback

升級與回滾

Arkheion upgrades by deploying a new module generation and rebuilding the governed relationships required by the cluster. Rollback uses historical records and podSnapshot data to restore a prior generation by logical contract ID. This is not a claim of universal zero-downtime behavior: availability depends on module design, transaction ordering and application-level handling during a change window.

Arkheion 通過部署新的模組 generation，並重建集群所需的受治理關係完成升級。回滾使用歷史記錄和 podSnapshot 資料，按邏輯合約 ID 恢復歷史 generation。這並不意味着任何場景都能無條件零停機：可用性仍取決於模組設計、交易順序以及變更窗口內的應用層處理。

07 Gas benchmark

Gas 基準測試

The benchmark compares Arkheion with Diamond (EIP-2535) using the same three-module DeFi system: PairStorage, FeeEngine and SwapEngine. Measurements were produced on a Hardhat 3 in-process network targeting Shanghai with optimizer runs set to 200. The source contracts, script, raw JSON data and report are included under [gas-benchmark/](#).

Benchmark 使用同一套三模組 DeFi 系統對比 Arkheion 與 Diamond (EIP-2535) : PairStorage、FeeEngine 和 SwapEngine。測試執行於 Hardhat 3 進程內網路, EVM 目標為 Shanghai, optimizer runs 為 200。源碼、腳本、原始 JSON 資料和報告均位於 [gas-benchmark/](#)。

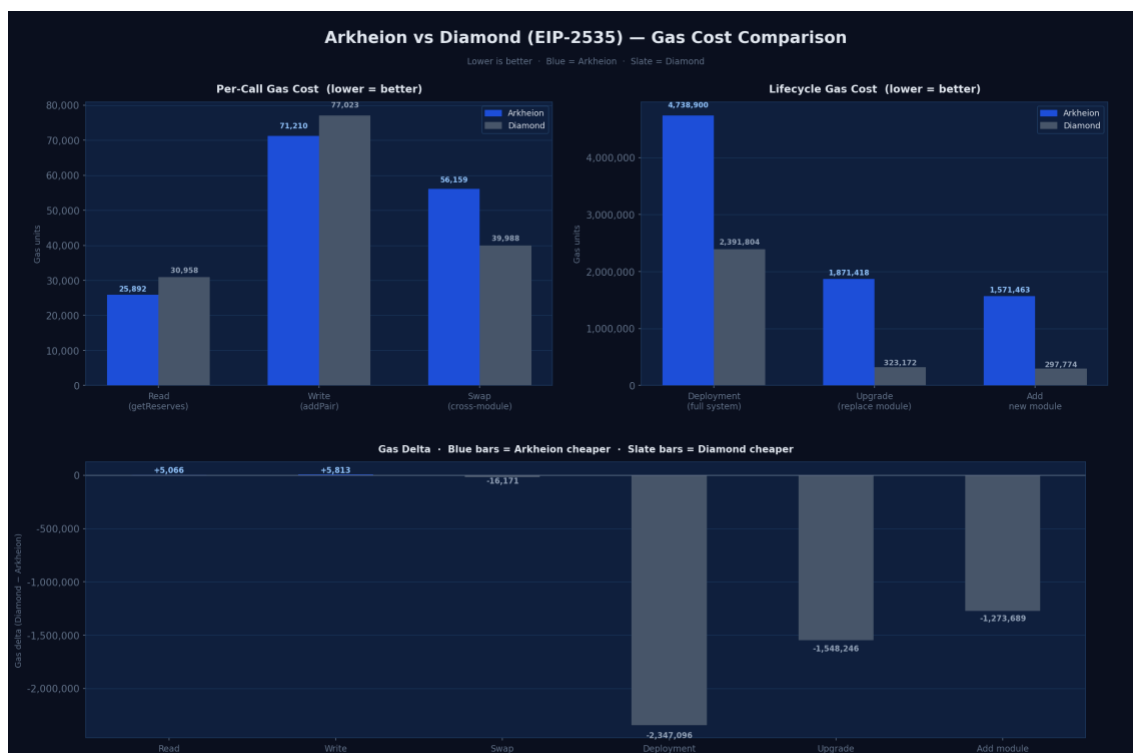


Figure 2. Arkheion vs Diamond gas cost comparison / 圖 2 : Arkheion 與 Diamond Gas 成本對比

Scenario	Arkheion	Diamond	Interpretation
Read / 讀取	25,892	30,958	Arkheion lower by 5,066 gas
Write / 寫入	71,210	77,023	Arkheion lower by 5,813 gas
Cross-module swap / 跨模組交換	56,159	39,988	Diamond lower by 16,171 gas
Deployment / 部署	4,738,900	2,391,804	Diamond lower lifecycle cost
Upgrade / 升級	1,871,418	323,172	Diamond lower lifecycle cost
Add module / 新增模組	1,571,463	297,774	Diamond lower lifecycle cost

Reading the result correctly

如何正確解讀結果

Arkheion is lower for direct reads and writes. Diamond is lower for cross-Pod swap and lifecycle operations. Arkheion accepts that cost in exchange for isolated module storage and explicit topology governance.

Arkheion 的直接讀寫成本較低；Diamond 的跨 Pod swap 與生命週期操作成本較低。Arkheion 接受此成本，以換取模組儲存隔離與顯式拓撲治理。

This is a tradeoff benchmark, not a universal cheaper-gas claim.

這是權衡型 benchmark，不是所有場景 Gas 都更低的主張。

08 Enterprise control plane

企業控制面

The enterprise edition extends the open-source contract cluster model into an institutional change-control plane. Its purpose is not to replace custody, compliance or core banking systems. It is the governed execution layer between an approved operating intent and the resulting on-chain state.

企業版將開源合約集群模型延伸為機構級變更控制面。它不是要取代託管、合規或核心銀行系統，而是位於已審批營運意圖與最終鏈上狀態之間的治理執行層。

Area	Enterprise direction	Current boundary
arkctl and task runtime arkctl 與任務執行時	Rust CLI, asynchronous task foundation, cancel and event-loop primitives	Foundations implemented; integration is ongoing 基礎已落地，整合仍在推進
Execution kernel 執行內核	ActionSpec -> Planner -> ExecutionPlan -> AtomicStep; preflight, idempotency, lock and ledger flow	Core skeleton exists; handlers and target semantics require hardening 核心骨架已存在，處理器與目標語意仍需強化
State source of truth 狀態事實源	PostgreSQL records for chains, clusters, plans, steps, evidence and reconciliation	Migration drafts and partial interfaces exist 已有遷移草案與部分介面
Governance and custody 治理與託管	master, collaborator and observer roles; committee workflow; Remote Signer boundary	Fail-closed permission and signer foundations exist; production injection is incomplete 已有 fail-closed 權限與簽名器基礎，生產接入尚未完成
Compiler evidence 編譯器證據	Standard JSON compilation, artifact manifests, ABI and storage-layout checks, proposal evidence	Compiler foundation exists; full validation pipeline remains roadmap work 已有編譯器基礎，完整驗證流程仍屬路線圖
Dashboard and API 控制台與 API	Organizations, clusters, nodes, operators, proposals, batches, versions, topology, audit and rollback jobs	Target product surface; not delivered as a production console 目標產品介面，尚未交付生產控制台

Large and small atomic operations

大型與小型原子操作

The enterprise design separates a complete institutional instruction from its verifiable chain steps. A large atomic operation represents the approved business change. Small atomic operations represent individual transactions or evidence-producing steps. This enables retry, compensation, rollback and error archiving without losing the original approval context.

企業設計將完整機構指令與可驗證鏈上步驟分離。大型原子操作代表已審批的業務變更；小型原子操作代表單筆交易或產生證據的步驟。如此可在不遺失原始審批語境的前提下，支援重試、補償、回滾與錯誤歸檔。

Enterprise status. The Rust control-plane foundation is under active development. This whitepaper distinguishes implemented foundations from the target production product.

企業版狀態：Rust 控制面基礎正在積極開發中。本文明確區分已落地基礎與目標生產產品。

09 Product editions and adoption

產品版本與採用路徑

Arkheion is positioned as next-generation financial infrastructure rather than a token-centered Web3 project. The open-source core lowers the cost of evaluation and invites developers to test the architecture directly. The enterprise path is a B2B software and services model built around operational controls, integration and support.

Arkheion 的定位是下一代金融基礎設施，而不是以代幣為中心的 Web3 項目。開源內核降低評估成本，並讓開發者可以直接驗證架構。企業化路徑是圍繞運營控制、集成和支持構建的 B2B 軟件與服務模式。

Open-source core	Enterprise direction
CLI workflow and EVM contracts CLI 流程與 EVM 合約	Managed SaaS or private deployment 託管 SaaS 或私有化部署
Declarative auto assembly 宣告式自動裝配	Policy engine and approval workflow 策略引擎與審批流
Topology, history and rollback 拓撲、歷史與回滾	Role integration, reporting and evidence export 角色集成、報告與證據導出
Local logs and project state 本地日誌與項目狀態	HSM or KMS and custody-system integration HSM 或 KMS 與託管系統集成
Developer-oriented interface 面向開發者的接口	Institutional support and implementation services 機構級支持與實施服務

Target adoption path

目標採用路徑

- Developer evaluation through the open-source CLI and reproducible samples.
開發者通過開源 CLI 和可復現樣板進行評估。
- Institutional proof of concept around one real contract cluster.
圍繞一套真實合約集群完成機構 PoC。
- Integration of approval, custody, audit and operational policies.
集成審批、託管、審計和運營策略。
- SaaS or private deployment according to institutional requirements.
根據機構要求提供 SaaS 或私有化部署。

10 Scope, limitations and roadmap

邊界、限制與路線圖

Current evidence

當前證據

- A working CLI and contract control-plane implementation exist in the repository.
倉庫中存在可執行的 CLI 與合約控制面實現。
- The 20-module DeFi sample has completed a local-chain end-to-end path including check, dry-run, auto assembly, graph or history inspection, upgrade and rollback.
20 模組 DeFi 樣板已完成本地鏈端到端路徑，包括 check、dry-run、auto assembly、graph 或 history 檢查、upgrade 與 rollback。
- The themed Gas benchmark is reproducible from the included source, scripts and raw results.
主題化 Gas benchmark 可通過倉庫內源碼、腳本和原始結果復現。

Known limits

已知限制

- Function-level dependency analysis is lightweight and convention-driven, not a formal verifier.
函數級依賴分析是基於約定的輕量分析，並非形式化驗證器。
- Cross-Pod calls and lifecycle orchestration introduce measurable Gas costs.
跨 Pod 呼叫與生命週期編排會引入可測量的 Gas 成本。
- Availability during upgrades depends on system design and operational procedure.
升級期間的可用性取決於系統設計與操作流程。
- Out-of-band topology changes can undermine local snapshot freshness and should be controlled operationally.
帶外拓撲變更可能破壞本地快照新鮮度，需要通過運營流程控制。

Roadmap priority 1: enhanced transaction system

路線圖重點一：增強型事務系統

The transaction system will be strengthened for institutional change control: approved operations should remain traceable, recoverable and reconcilable across their lifecycle.

事務系統將面向機構級變更控制持續強化：已審批操作應在完整生命週期內保持可追溯、可恢復與可對帳。

- Improve transaction planning, execution records and policy-controlled recovery.
完善事務計畫、執行記錄與受策略控制的恢復能力。
- Strengthen chain-to-database synchronization and reconciliation.
強化鏈上狀態與資料庫之間的同步及對帳。

Roadmap priority 2: compiler integration

路線圖重點二：編譯器整合

Compiler integration will provide stronger pre-deployment assurance and clearer evidence for institutional review.

編譯器整合將提供更強的部署前保障，以及更清晰的機構覆核證據。

- Add artifact-driven compatibility and deployability checks for lifecycle proposals.
為生命週期提案加入由編譯產物驅動的相容性與可部署性檢查。
- Bind compiler validation results into the audit evidence chain.
將編譯驗證結果納入審計證據鏈。

11 Getting started and references

快速開始與參考資料

Minimal developer path

最小開發者路徑

```
npm install
node cli/index.js --help
arkheion init
arkheion cluster init
arkheion cluster check
arkheion cluster auto --dry-run
arkheion cluster auto
```

Developers evaluating Arkheion should begin with the CLI help, the bilingual README, the 20-module sample under demo/realtest-20/, and the benchmark under gas-benchmark/. The architecture is intended to be inspected and tested, not accepted as a narrative claim.

評估 Arkheion 的開發者應從 CLI help、雙語 README、demo/realtest-20/ 下的 20 模組樣板以及 gas-benchmark/ 入手。該架構的目標是接受檢查和測試，而不是依賴敘事性主張。

Repository references

倉庫參考資料

- [1] README.md and README.zh-CN.md - project overview and CLI entry path.
[1] README.md 與 README.zh-CN.md - 項目總覽與 CLI 入口。
- [2] libs/commands/cluster/auto.js - staged declarative assembly implementation.
[2] libs/commands/cluster/auto.js - 分階段宣告式裝配實現。
- [3] libs/commands/cluster/upgrade.js and rollback.js - version governance and recovery paths.
[3] libs/commands/cluster/upgrade.js 與 rollback.js - 版本治理與恢復路徑。
- [4] gas-benchmark/GAS_REPORT.md and gas_results.json - reproducible benchmark report and raw results.
[4] gas-benchmark/GAS_REPORT.md 與 gas_results.json - 可復現 benchmark 報告與原始資料。
- [5] documents/workspace/archieve/E2E_TEST_REPORT_2026-03-22.md - local-chain E2E evidence and observed issues.
[5] documents/workspace/archieve/E2E_TEST_REPORT_2026-03-22.md - 本地鏈 E2E 證據與觀察項。

Contact

聯絡方式

- Email: youtianshuo90@gmail.com
電子郵件：youtianshuo90@gmail.com
- GitHub: Steve65535
GitHub：Steve65535